

PROJECT: PERFORM SCAN ON DAM VULNERABILITY WEB APPLICATION AND RECOMMEND REMEDIATION

STEPS TAKEN TO SCAN THE DVWA

BASICS:

- a. XAMPP was installed and running in colour green (Apache & MySQL).
- b. DVWA was installed in htdocs and configured correctly.
- c. DVWA security level was set to low.
- d. OWASP ZAP, A vulnerability scanner was installed.

Scanning Steps Using OWASP ZAP

- v I Started XAMPP
- v I Opened XAMPP Control Panel
- v I Started Apache and MySQL
- v I Visited <http://localhost/dvwa/> to confirm DVWA was running, and it was.

Login to DVWA

- v I used the Default credentials, viz:
- v Username: admin
- v Password: password
- v I Navigated to DVWA Security and set security level to low

Opening the OWASP ZAP

- v I Launched the ZAP tool
- v I Chose "Manual Explore" or "Automated Scan" on the two instances..

Manual Explore (Recommended)

- v In ZAP, I went to :
- v File → New Session
- v Tools → Options → Under Local Proxy, confirm ZAP is listening on port 8080
- v Configuration of the browser:
- v I Set the proxy to 127.0.0.1 port 8080
- v I Visited <http://localhost/dvwa/> in my browser (ZAP intercepted the traffic)

Spider the Site

- v In ZAP, I right-clicked on the <http://localhost> tree item → Attack → Spider Site
- v This lets ZAP discover all pages and parameters

Active Scan

- v I Right-clicked the same item again → Attack → Active Scan
- v Selected the context/scope and let ZAP perform the scan

Review Alerts

- v After scanning, I went to the "Alerts" tab to view:
- v SQL Injection
- v XSS (Reflected/Stored)
- v Command Injection
- v Security Misconfiguration

PRECAUTIONARY STEPS:

- v Ensure DVWA security was set to 'Low' for accurate scanning.

v The Scan was carried out on a localhost

v I Avoided using these tools on real-world websites unless authorized.



MANUAL TESTING PROCEDURE:

1. I Prepared my Environment

v Start XAMPP

Start Apache and MySQL

Open DVWA in your browser:

<http://localhost/dvwa/>

Login using:

Username: admin

Password: password

v **Setting Security Level**

Go to DVWA Security

Set to Low (for testing purposes)

v **Start OWASP ZAP**

Open ZAP

Create a new session (optional)

2. Configure Browser to Use ZAP Proxy

v **Set Browser Proxy to:**

HTTP Proxy: 127.0.0.1

Port: 8080

OR use ZAP's browser:

Go to:

Tools → Options → Local Proxy

(I Verified it's running on port 8080)

Launch browser from ZAP:

File → Launch Browser

3. Manually Explore the Web App (DVWA)

With proxy on, I browsed through DVWA normally:

Login

Visit different vulnerability modules (e.g., Command Injection, SQL Injection)

Submit different inputs in forms

ZAP captured and recorded all requests/responses under:

Sites tab (left pane)

History tab (bottom pane)

4. Analysis of Requests/Responses

At the History tab:

I Clicked on each request

I Examined:

Request headers

Parameters sent

Response codes

Cookies

Server responses

I looked for:

Suspicious reflections (e.g., XSS vectors)

SQL error messages

Parameters not sanitized

5. Manual Attack Techniques

Reflected XSS Testing

I Found a form/ URL parameter

I Modified input to: `<script>alert(1)</script>`

I Checked if it appeared in the response (rendered)

SQL Injection Testing

Input: `' OR 1=1-- or '`

I Looked for SQL errors in the response

I also tried login forms in DVWA's SQL Injection module

Command Injection Testing

Input: ; ping 127.0.0.1 ls

I Checked response for command execution output

Tools Used:

OWASP ZAP (Manual Proxy)

Browser (configured to use ZAP Proxy)

DVWA (Security level was set to "low")

Manual Testing Procedure for Security Misconfiguration

1. I Prepared my Environment thus:

Start XAMPP → Apache & MySQL

Open <http://localhost/dvwa/>

Log in as admin / password

Start ZAP and configure browser proxy to 127.0.0.1:8080

2. Passive Scanning with ZAP (initial check)

ZAP automatically captured and analyzed requests as I browsed DVWA.

I Checked the following tabs:

Sites → Right-click → "Show Alerts"

Alerts Tab → Look for:

Missing Security Headers

Information Disclosure

Directory Listing

Insecure Cookies

Manual Verification Checklist

Go to History tab → click any request

Check Response headers:

der	ected	onfiguration
Content-Type-Options	niff	ing
ame-Options	IY or SAMEORIGIN	ing (Clickjacking risk)
Content-Security-Policy	tricts sources	ing (XSS risk)
Strict-Transport-Security	es HTTPS	ing or misconfigured

Test: I Browsed DVWA and inspect if headers are present
In ZAP → Right-click request → "Resend" → View headers tab

I tested by Visiting:

<http://localhost/dvwa/>

Check:

DVWA default login still works? (admin / password)

Login failed, so the following errors were revealed:

Invalid username (reveals account)

Database connection error (stack traces)

Cause errors manually:

Visit a non-existent page:
<http://localhost/dvwa/weep.php>

Submit invalid data in forms

The page returned full PHP errors (`mysqli_sql_exception`), which signified that it was misconfigured.

4. ZAP Active Scan (for Confirmation)

I Right-clicked on <http://localhost> → Attack → Active Scan

Review Alerts tab for:

Missing Headers

Default Server Banners (e.g., Apache/2.4.58)

Sensitive files exposed

5. Manual File/Path Discovery

I tried visiting common admin paths:

<http://localhost/dvwa/phpinfo.php>

<http://localhost/dvwa/config>

<http://localhost/.git/>

There was a return of contents, which signified that there's a misconfiguration.

6. I Used ZAP's Manual Tools

Right-click a request in History tab → and Used the following tools:

Tool	Use
Request Editor	Manually change requests and resend them
Fuzzer	Fuzz parameters with payloads (XSS, SQLi, etc.)
Breakpoints	Intercept and modify requests before sending
Search	Look for keywords in responses (like "error", "alert")

7. Check the "Alerts" Tab

As you manually explore, ZAP may automatically detect:

Missing security headers

Potential vulnerabilities

These show up in the Alerts tab with risk ratings

8. I Saved The Session & Prepared the requisite Report

File → Save Session As to keep your test logs

Export HTML report:

Report → I Generated the Report

Summary

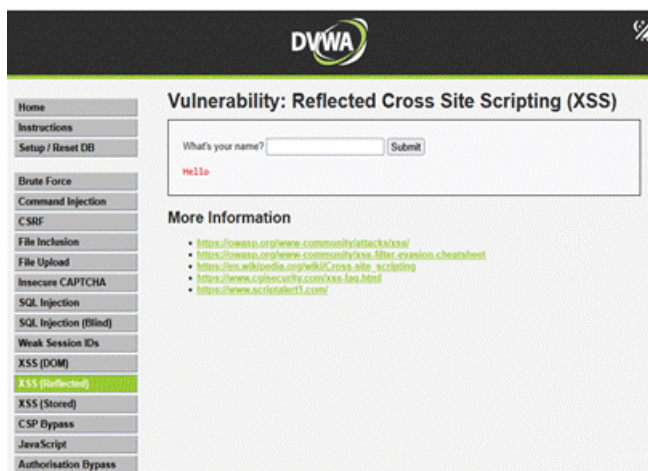
ase :tion

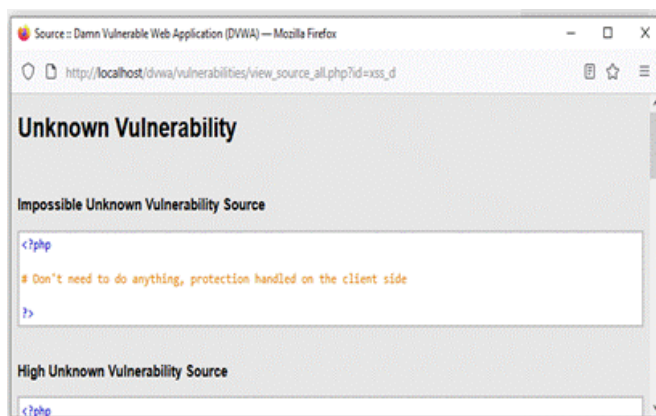
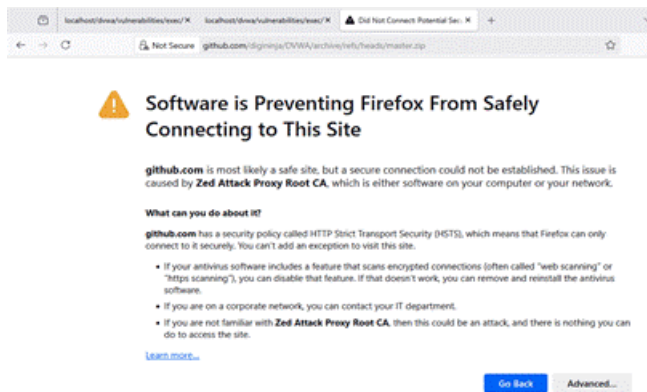
Setup	Start DVWA & ZAP, configure browser proxy
Explore	Browse DVWA through ZAP
Analyze	Review requests/responses in History
Test	Select payloads manually
Tools	Use breakpoints, fuzzer, manual request editor
Report	View Alerts & export report

C. Vulnerability Report Table

Vulnerability	Severity	Description	Affected Component
Missing Security Headers	Medium	The application does not implement standard HTTP security headers like Frame-Options, X-Content-Type-Options, and Content-Security-Policy, increasing the risk of XSS, clickjacking, and MIME sniffing attacks.	Web Server (Apache)
Directory Listing Enabled	High	The /dvwa/hackable/uploads/ directory is publicly accessible and lists uploaded files, potentially exposing sensitive or malicious files to attackers.	Web Server Configuration

Command Injection	Critical	User input in the "Command Injection" module is executed directly in the system shell, allowing arbitrary command execution such as <code>whoami</code> .	<code>index.php</code> (DVWA Vulnerability Module)
Reflected XSS	High	Unvalidated input is reflected back to the user without sanitization, allowing JavaScript execution via payloads like <code><script>alert(1)</script></code> .	<code>s.php</code> or <code>wa/vulnerabilities/xss_r/</code>
SQL Injection	Critical	The user ID input field is vulnerable to SQL injection. An attacker can bypass authentication or extract database data using payloads like <code>' OR 1=1 --</code> .	<code>lnerabilities/sqli/</code>
Verbose Error Message	Low	Valid input in the login form triggers detailed SQL error messages, exposing stack traces and database structure.	<code>cludes/dvwaPage.inc.php</code>
Default Credentials	Medium	DVWA is accessible with default credentials (admin / password), which can be easily guessed by attackers.	Authentication System (DVWA login)





D. Vulnerability Remediation Table

Vulnerability

Remediation Strategy

Missing Security Headers	Configure the web server (e.g., Apache) to include essential security headers: • Add X-Frame-Options: DENY to prevent clickjacking • Add X-Content-Type-Options: nosniff to prevent MIME sniffing • Add a Content-Security-Policy to control resource loading • Add Strict-Transport-Security if HTTPS is used
Directory Listing Enabled	Disable directory browsing in the Apache config or .htaccess: Options -Indexes in the directory block to prevent file listings
Command Injection	• Sanitize and validate user input strictly (use whitelisting) • Do not pass input directly to system commands • Use safe APIs instead of exec() or system() functions • Employ Web Application Firewalls (WAF) to detect/block such input
Reflected XSS	• Properly escape and encode output • Sanitize input using a whitelist approach • Implement Content Security Policy (CSP) • Use modern frameworks with built-in XSS protection
SQL Injection	• Use prepared statements (parameterized queries) • Avoid building SQL queries using string concatenation • Validate and sanitize all user inputs • Use ORM frameworks where possible

Verbose Error
Message

- Turn off error reporting in production environments (e.g., `display_errors = Off` in `php.ini`)
- Log errors to a secure file instead
- Display generic error messages to users

Default Credentials

- Change all default credentials immediately after installation
- Enforce strong password policies
- Implement account lockout and two-factor authentication where feasible

Recommendations for Implementation

- v Secure Configuration Review: Review all server and app configurations regularly.
- v Code Review: Audit and refactor legacy code to ensure input validation.
- v User Awareness: Educate developers and administrators on secure coding and deployment practices.
- v Use Security Tools: Employ scanners, SAST tools, and WAFs in your CI/CD pipeline.

Observations

- v Numerous Common Vulnerabilities Exist.

DVWA intentionally contains insecure code that exposes it to real-world vulnerabilities such as SQL Injection, Cross-Site Scripting (XSS), Command Injection, and Security Misconfiguration.

- v Security Misconfigurations Are Easy to Overlook.

Important HTTP security headers were missing, and directory listing was enabled—these are often overlooked in real applications but provide significant attack surfaces.

- v Error Messages Leak Sensitive Information.

Some modules returned full error messages, including stack traces and database errors, which could aid attackers during reconnaissance.

v ZAP Accurately Identifies Vulnerabilities.

ZAP was able to identify missing security headers and reflected inputs without active scanning, proving its usefulness for both passive and active assessments.

v Default Credentials Still Work.

Default credentials (admin/password) were accepted, highlighting how poor credential hygiene can be a major risk.

v Manual Testing Reveals Deeper Context.

Manual payload testing allowed precise control, making it easier to observe how the application processes input, especially in command and SQL injection cases.

v My system's security setting had to be temporarily disabled as it kept removing downloads. I had to use Git bash to get the job done.

Lessons Learned

v Manual Testing Complements Automated Scans.

While ZAP automates vulnerability detection, manual testing is essential for understanding how each vulnerability works and how to exploit it safely in a controlled environment.

v Input Validation Is Critical.

Most vulnerabilities existed because user input was not validated or sanitized. This underscores the importance of proper input handling in secure coding.

v Server Configuration Affects Security.

Even if code is secure, server misconfigurations (like directory listing or missing headers) can expose the application. Security is multi-layered.

v Error Handling Should Be Non-Verbose in Production.

Verbose error messages can leak sensitive information and must be disabled in production environments. Only generic messages should be shown to users.

v Security Is Not a One-Time Task.

This exercise demonstrates that secure application development requires ongoing testing, patching, and configuration review.

v Understanding the Attacker's Perspective Is Valuable.

Learning how to manually test vulnerabilities gives a deeper understanding of how attackers think, which improves your ability to defend against them.

v Training in Safe Environments Is Essential.

DVWA provides a legal and controlled environment for practicing ethical hacking skills, making it an ideal platform for learning without violating any laws or policies.

SCAN POLICY

1. SQL Injection

URL: <http://localhost/dvwa/vulnerabilities/sqli/>

Risk: High

Confidence: High

Parameter: id (or whatever parameter was used)

Attack: 1' OR '1'='1

Evidence: Welcome admin (or a string returned showing successful injection)

CWE ID: 89 (CWE-89: SQL Injection)

WASC ID: 19 (WASC-19: Structured Query Language Injection)

Description:

The application is vulnerable to SQL Injection via the id parameter. Input is not sanitized and SQL queries are constructed insecurely.

Other Info:

This can lead to unauthorized access or database data exposure.

Solution:

Use parameterized queries or stored procedures. Validate and sanitize all user inputs.

2. Cross-Site Scripting (XSS)

URL: http://localhost/dwva/vulnerabilities/xss_r/

Risk: High

Confidence: High

Parameter: name

Attack: `<script>alert('XSS')</script>`

Evidence: alert('XSS') shown in browser

CWE ID: 79 (Cross-site Scripting)

WASC ID: 8 (Cross-site Scripting)

Description:

The application reflects unsanitized user input in the response, allowing malicious JavaScript to be executed in the user's browser.

Other Info:

This can be exploited to steal session cookies, redirect users, or perform actions on behalf of the victim.

Solution:

Encode all user input before reflecting it to the browser. Use modern frameworks and libraries that automatically escape output.

3. Command Injection

URL: `http://localhost/dvwa/vulnerabilities/exec/`

Risk: Critical

Confidence: High

Parameter: ip

Attack: `127.0.0.1; cat /etc/passwd`

Evidence: Contents of `/etc/passwd` or command output in the response

CWE ID: 77 (Command Injection)

WASC ID: 31 (OS Commanding)

Description:

User input is directly passed to a system shell command, which can allow attackers to execute arbitrary system-level commands.

Other Info:

Successful exploitation may lead to full system compromise.

Solution:

Avoid using system calls with user input. Validate input strictly and use safe APIs that do not invoke a shell.

4. Security Misconfiguration

URL: <http://localhost/dvwa/>

Risk: Medium

Confidence: Medium

Parameter: (leave blank unless specific)

Attack: (N/A unless specific misconfig trigger is found)

Evidence: Application is running in default configuration, default credentials may still be enabled.

CWE ID: 933 (Security Misconfiguration)

WASC ID: 14 (Improper Server Configuration)

Description:

The application or web server is using insecure default configurations, which can expose it to attack.

Other Info:

Example: default XAMPP installation exposes phpinfo() or DVWA setup pages without authentication.

Solution:

Harden server and application configurations. Remove unnecessary services, disable debug/info pages, change default passwords.

Policy Name:

"DVWA Core Vulnerabilities - Injection & Misconfig"

Description:

Covers key attack vectors including:

SQL Injection (SQLi)

Cross-Site Scripting (XSS)

Command Injection

Security Misconfiguration

You can create this as a custom scan policy by:

Going to Tools > Options > Scan Policy in ZAP.

Click Add, name it as above.

Enable rules under each category (Injection, Input Validation, Server Security, etc.) that relate to the vulnerabilities you're targeting.